

Informe Técnico: Algoritmo para Evaluar la Viabilidad Técnica de un Proyecto Estudiantil

Jhon Mario Guamanzar Sierra

Servicio Nacional de Aprendizaje (SENA)

Programa: Análisis y Desarrollo de Software (Ficha: 3321349)

Instructor Antony Reynel Botello Herrera

Agosto de 2026

Tabla de Contenido

Resumen	4
Introducción	5
Análisis del Problema	5
Entradas, Procesos y Salidas	6
Criterios de Evaluación Aplicados	7
Desarrollo e Implementación en Lenguaje Python	8
Estructura del Código y Tipificación Dinámica en Python	8
Captura Interactiva y Procesamiento Aritmético	9
Estructura de Decisión Compuesta y Control de Flujo	9
Pruebas de Ejecución en Consola Python	10
Desarrollo y Representación en PSeInt	11
Declaración del Algoritmo y Variables en Pseudocódigo	12
Solicitud de Datos y Cálculo del Promedio en Pseudocódigo	12
Condición de Decisión en Pseudocódigo	13
Representación Gráfica mediante Diagrama de Flujo	14
Diagrama de Flujo en PSeInt	14
Pruebas de Escritorio y Ejecución en PSeInt	15
Prueba de Escritorio 1: Proyecto con Requerimiento de Ajustes	15
Prueba de Escritorio 2: Proyecto con Resultado Viable	16
Conclusiones	17
Referencias	18

Lista de Tablas

Tabla 1. Matriz de Entradas, Procesos y Salidas del Algoritmo de Viabilidad Técnica	6
Tabla 2. Criterios Técnicos y Pedagógicos Aplicados en el Informe	7

Lista de Figuras

Figura 1. Entorno de Desarrollo VS Code con el Código Fuente en Python (ejercicio_pseint.py)	8
Figura 2. Evidencia de Ejecución en Python 1 con Caso: Proyecto No Es Viable	10
Figura 3. Evidencia de Ejecución en Python 2 con Caso: Proyecto Viable	11
Figura 4. Declaración del Algoritmo y Definición de Variables en PSeInt	12
Figura 5. Instrucciones de Solicitud y Lectura de Datos Iniciales en PSeInt	12
Figura 6. Lectura del Puntaje de Arquitectura y Cálculo Aritmético del Promedio en PSeInt ...	13
Figura 7. Estructura Condicional Compuesta y Generación de Mensajes en PSeInt	13
Figura 8. Diagrama de Flujo de la Solución Algorítmica en PSeInt	14
Figura 9. Evidencia de Ejecución 1 en PSeInt: Proyecto Requiere Ajustes	15
Figura 10. Evidencia de Ejecución 2 en PSeInt: Proyecto Viable Técnicamente	16

Resumen

El presente informe técnico desarrolla una solución algorítmica estructurada para evaluar la viabilidad técnica inicial de un proyecto de software formativo en el ámbito estudiantil. La metodología planteada se fundamenta en la cuantificación de los criterios críticos de calidad de software: el nivel de cumplimiento de los requisitos técnicos, la solidez de la arquitectura de software propuesta y la conformidad en las pruebas de software. A partir de dichos valores numéricos, el sistema computa el promedio aritmético ponderado y somete los datos a una estructura de decisión lógica compuesta, la cual determina si el proyecto califica como viable técnicamente o si requiere ajustes previos a la fase de construcción. El documento detalla el análisis sistemático de entradas, procesos y salidas, la implementación computacional en lenguaje de programación Python desarrollada en el entorno Visual Studio Code, el desarrollo del pseudocódigo estructurado, la diagramación de flujo estandarizada y las evidencias de verificación mediante pruebas de escritorio y ejecución en terminal tanto en Python como en PSeInt.

Palabras clave: algoritmo, Python, pseudocódigo, PSeInt, diagrama de flujo, viabilidad técnica, lógica condicional.

Informe Técnico: Algoritmo para Evaluar la Viabilidad Técnica de un Proyecto Estudiantil

Introducción

La planificación rigurosa de un proyecto de software constituye una fase determinante para establecer si las condiciones técnicas, conceptuales y metodológicas garantizan un desarrollo exitoso antes de comprometer recursos en la etapa de construcción. En el marco de la formación profesional en Análisis y Desarrollo de Software del Servicio Nacional de Aprendizaje [SENA] (2026), se propone una actividad pedagógica orientada a consolidar la apropiación práctica de la lógica proposicional, las estructuras de control condicionales, la notación algorítmica en pseudocódigo y la implementación computacional en lenguajes modernos de programación como Python.

El problema técnico formulado requiere capturar de manera interactiva el nombre del proyecto estudiantil y las calificaciones cuantitativas continuas en una escala de 0 a 100 puntos, correspondientes al cumplimiento de requisitos técnicos, viabilidad de la arquitectura y evaluación de pruebas. Con estos valores, el algoritmo calcula el promedio aritmético y evalúa una regla de decisión compuesta. Para que un proyecto sea dictaminado como técnicamente viable, debe satisfacer simultáneamente dos criterios: obtener un promedio global igual o superior a 75 puntos y registrar calificaciones estrictamente mayores a 60 puntos en cada uno de los componentes individuales evaluados. Este mecanismo de control integral previene que un promedio global satisfactorio enmascare deficiencias críticas en alguno de los aspectos fundamentales del proyecto (Aguilar Joyanes, 2008).

Análisis del Problema

El propósito fundamental del algoritmo consiste en automatizar y estandarizar la evaluación preliminar de viabilidad técnica, minimizando la subjetividad en la toma de decisiones. Para estructurar la solución computacional, se aplicó el método clásico de descomposición de sistemas propuesto por Vásquez (2012), identificando de manera inequívoca

las entradas de información, las operaciones aritméticas y lógicas de procesamiento, y las salidas de resultados entregadas al usuario.

Entradas, Procesos y Salidas

La interacción y el flujo de datos del sistema algorítmico se sintetizan en la Tabla 1, donde se delimitan los componentes del modelo de procesamiento de información.

Tabla 1

Matriz de Entradas, Procesos y Salidas del Algoritmo de Viabilidad Técnica

Elemento	Descripción
Entradas	Nombre del proyecto estudiantil (tipo carácter/string), puntaje de cumplimiento de requisitos técnicos (numérico real de 0 a 100), puntaje de viabilidad de la arquitectura (numérico real de 0 a 100) y puntaje de pruebas de software (numérico real de 0 a 100).
Procesos	Lectura interactiva y validación de rangos, cálculo algebraico del promedio aritmético y evaluación condicional compuesta mediante operadores lógicos conjuntivos.
Salidas	Emisión de mensaje formal indicando si el proyecto es viable técnicamente o si no es viable (requiere ajustes), acompañado del nombre del proyecto y el promedio final obtenido.

Nota. Componentes funcionales del modelo computacional implementado para la evaluación de viabilidad.

En términos de lógica proposicional, la regla de decisión se modela formalmente mediante la conjunción booleana: `promedio >= 75 Y puntajeRequisitos > 60 Y puntajeArquitectura > 60 Y puntajePruebas > 60`. Si todas las proposiciones elementales son verdaderas, la expresión lógica global se evalúa como verdadera y se dictamina la viabilidad técnica; si al menos una de las proposiciones resulta falsa, el flujo lógico se deriva hacia la alternativa que indica que el proyecto no es viable.

Criterios de Evaluación Aplicados

El diseño e implementación del algoritmo integran de manera transversal los lineamientos técnicos, las directrices pedagógicas del SENA y los estándares de calidad en el desarrollo de software (Sierra, 2000). En la Tabla 2 se detallan los criterios de evaluación contemplados y su correspondencia dentro del presente documento.

Tabla 2

Criterios Técnicos y Pedagógicos Aplicados en el Informe

Criterio	Aplicación en el Informe
Documento con análisis y representación	Incluye análisis formal del problema, descomposición funcional, código fuente en Python, pseudocódigo estructurado, diagrama de flujo estandarizado y evidencias de ejecución.
Entradas de datos	Identifica, valida y tipifica adecuadamente el nombre del proyecto y las calificaciones continuas solicitadas.
Procesos matemáticos	Calcula con precisión el promedio aritmético de los puntajes ingresados mediante operadores algebraicos.
Estructuras condicionales	Implementa una bifurcación lógica compuesta con operadores de conjunción (and / Y) para evaluar múltiples restricciones simultáneas.
Salidas y mensajes	Construye mensajes dinámicos, claros y contextualizados para las condiciones de viabilidad y no viabilidad técnica.
Representación de la solución	Incorpora script ejecutable en Python 3.x documentado en Visual Studio Code, código fuente en pseudocódigo y diagrama de flujo en PSeInt.
Presentación y ortografía	Aplica rigurosamente las Normas APA 7. ^a edición, redacción técnica académica, tablas y figuras numeradas con notas y referencias bibliográficas normalizadas.

Nota. Relación de criterios pedagógicos e ingenieriles verificados en el desarrollo de la evidencia formativa.

Desarrollo e Implementación en Lenguaje Python

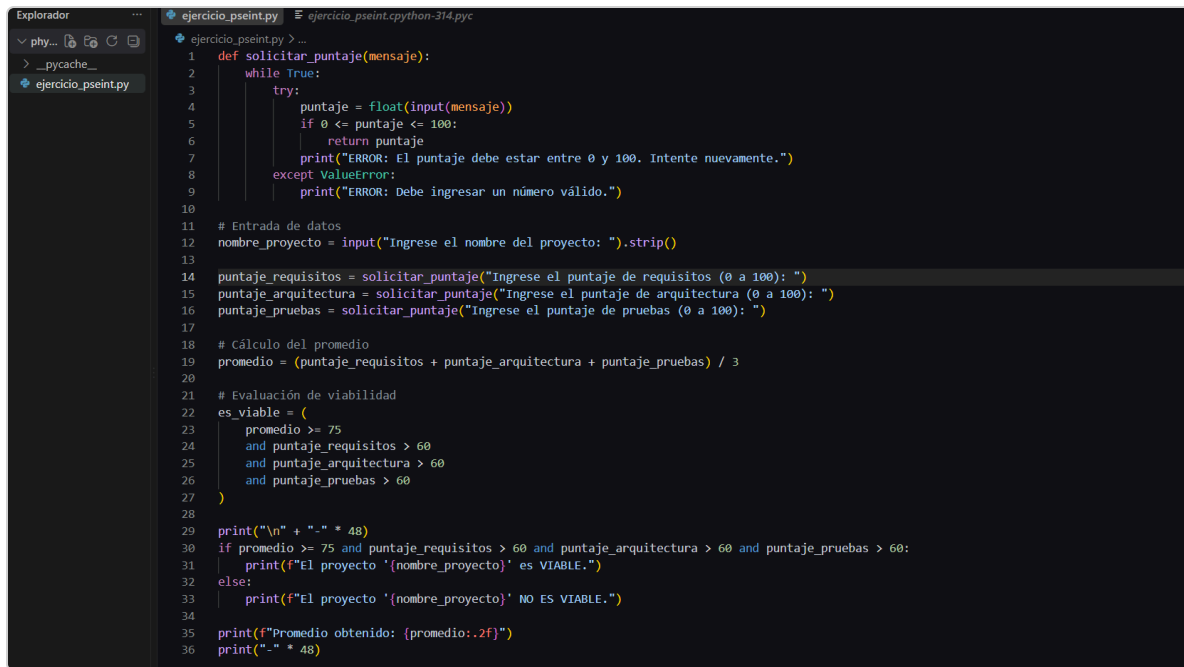
Como primera instancia de implementación computacional, el algoritmo fue codificado en el lenguaje de programación Python (versión 3.x) en el entorno de desarrollo Visual Studio Code. Python ofrece una sintaxis moderna, tipado dinámico y funciones modulares que permiten robustecer la captura de datos con control de excepciones y validaciones de rango.

Estructura del Código y Tipificación Dinámica en Python

El código del archivo `ejercicio_pseint.py` implementa una función modular denominada `solicitar_puntaje(mensaje)`. Esta función contiene un bucle interactivo `while True` con control de excepciones `try-except ValueError`, el cual asegura que el usuario ingrese exclusivamente valores numéricos válidos en el intervalo cerrado de 0 a 100 puntos. Posteriormente, se realiza la captura del nombre del proyecto estudiantil mediante `input().strip()` y la lectura de los tres puntajes de evaluación: requisitos, arquitectura y pruebas. En la Figura 1 se presenta la captura directa del código fuente desarrollado en Visual Studio Code.

Figura 1

Entorno de Desarrollo VS Code con el Código Fuente en Python (ejercicio_pseint.py)



```

1 def solicitar_puntaje(mensaje):
2     while True:
3         try:
4             puntaje = float(input(mensaje))
5             if 0 <= puntaje <= 100:
6                 return puntaje
7             print("ERROR: El puntaje debe estar entre 0 y 100. Intente nuevamente.")
8         except ValueError:
9             print("ERROR: Debe ingresar un número válido.")
10
11 # Entrada de datos
12 nombre_proyecto = input("Ingrese el nombre del proyecto: ").strip()
13
14 puntaje_requisitos = solicitar_puntaje("Ingrese el puntaje de requisitos (0 a 100): ")
15 puntaje_arquitectura = solicitar_puntaje("Ingrese el puntaje de arquitectura (0 a 100): ")
16 puntaje_pruebas = solicitar_puntaje("Ingrese el puntaje de pruebas (0 a 100): ")
17
18 # Cálculo del promedio
19 promedio = (puntaje_requisitos + puntaje_arquitectura + puntaje_pruebas) / 3
20
21 # Evaluación de viabilidad
22 es_viable = (
23     promedio >= 75
24     and puntaje_requisitos > 60
25     and puntaje_arquitectura > 60
26     and puntaje_pruebas > 60
27 )
28
29 print("\n" + "-" * 48)
30 if promedio >= 75 and puntaje_requisitos > 60 and puntaje_arquitectura > 60 and puntaje_pruebas > 60:
31     print(f"El proyecto '{nombre_proyecto}' es VIABLE.")
32 else:
33     print(f"El proyecto '{nombre_proyecto}' NO ES VIABLE.")
34
35 print(f"Promedio obtenido: {promedio:.2f}")
36 print("-" * 48)

```

Nota. Captura de pantalla del entorno Visual Studio Code donde se observa la función de validación de datos, la captura de entradas y la estructura de decisión en Python.

Captura Interactiva y Procesamiento Aritmético

Una vez recolectadas las calificaciones en memoria, el programa efectúa el procesamiento matemático computando la suma de los tres puntajes y dividiendo el resultado entre tres para obtener el promedio aritmético global: `promedio = (puntaje_requisitos + puntaje_arquitectura + puntaje_pruebas) / 3`. Esta fórmula garantiza la ponderación equitativa de cada componente técnico.

Estructura de Decisión Compuesta y Control de Flujo

La evaluación de viabilidad técnica se modela a través de la conjunción booleana con el operador `and`: `promedio >= 75 and puntaje_requisitos > 60 and puntaje_arquitectura > 60 and puntaje_pruebas > 60`. Esta expresión lógica compuesta se evalúa en una sentencia `if-else`. Si todas las proposiciones son verdaderas, el

sistema imprime el mensaje afirmativo de viabilidad; si al menos un criterio no supera el umbral, se ejecuta la rama alternativa indicando que el proyecto no es viable.

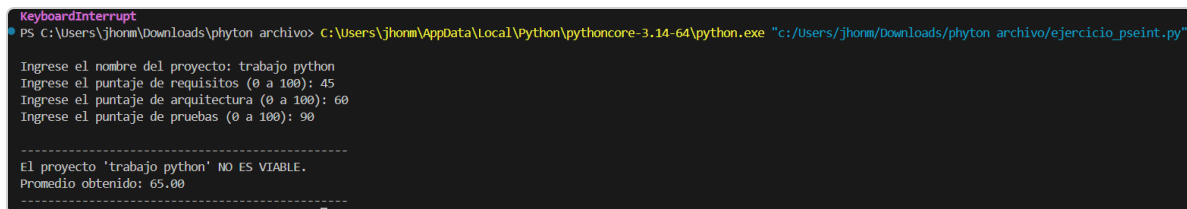
Pruebas de Ejecución en Consola Python

Para validar la precisión del algoritmo en Python, se realizaron dos pruebas de ejecución en la terminal PowerShell con los escenarios de prueba representativos.

En el primer escenario experimental, se evaluó el proyecto 'trabajo python' con calificaciones de 45 puntos en requisitos, 60 puntos en arquitectura y 90 puntos en pruebas. Aunque el puntaje de pruebas fue sobresaliente, el promedio obtenido fue de 65.00 puntos y los requisitos no superaron el mínimo de 60. Como se evidencia en la Figura 2, la terminal dictaminó correctamente que el proyecto NO ES VIABLE.

Figura 2

Evidencia de Ejecución en Python 1 con Caso: Proyecto No Es Viable



```
KeyboardInterrupt
PS C:\Users\jhorn\Downloads\phyton archivo> C:\Users\jhorn\AppData\Local\Python\pythoncore-3.14-64\python.exe "c:/Users/jhorn/Downloads/phyton archivo/ejercicio_pseint.py"

Ingrese el nombre del proyecto: trabajo python
Ingrese el puntaje de requisitos (0 a 100): 45
Ingrese el puntaje de arquitectura (0 a 100): 60
Ingrese el puntaje de pruebas (0 a 100): 90

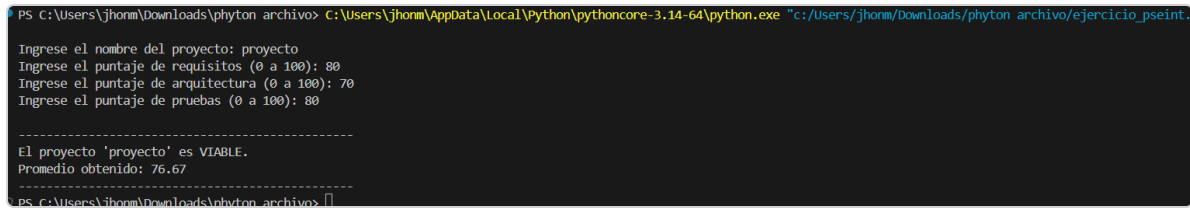
-----
El proyecto 'trabajo python' NO ES VIABLE.
Promedio obtenido: 65.00
-----
```

Nota. Captura de pantalla de la terminal PowerShell donde se valida el rechazo del proyecto al obtener un promedio de 65.00 y no cumplir con los umbrales individuales mínimos.

En el segundo escenario experimental, se evaluó el proyecto 'proyecto' con calificaciones de 80 puntos en requisitos, 70 puntos en arquitectura y 80 puntos en pruebas. El promedio alcanzado fue de 76.67 puntos y cada puntaje individual superó estrictamente los 60 puntos. Tal como se exhibe en la Figura 3, al cumplirse todas las condiciones lógicas, la consola emitió el mensaje de aprobación confirmando que el proyecto es VIABLE.

Figura 3

Evidencia de Ejecución en Python 2 con Caso: Proyecto Viable



```
PS C:\Users\jhoni\Downloads\phyton archivo> C:\Users\jhoni\AppData\Local\Python\pythoncore-3.14-64\python.exe "c:/Users/jhoni/Downloads/phyton archivo/ejercicio_pseint.
Ingrese el nombre del proyecto: proyecto
Ingrese el puntaje de requisitos (0 a 100): 80
Ingrese el puntaje de arquitectura (0 a 100): 70
Ingrese el puntaje de pruebas (0 a 100): 80
-----
El proyecto 'proyecto' es VIABLE.
Promedio obtenido: 76.67
-----
PS C:\Users\jhoni\Downloads\phyton archivo>
```

Nota. Captura de pantalla de la terminal PowerShell evidenciando la aprobación de viabilidad técnica al alcanzarse un promedio de 76.67 puntos y cumplirse todas las restricciones.

Desarrollo y Representación en PSeInt

Complementando la solución en Python, se estructuró la versión formal en pseudocódigo y diagrama de flujo mediante el software pedagógico PSeInt, verificando la correspondencia entre la notación algorítmica básica y los lenguajes de programación de alto nivel.

Declaración del Algoritmo y Variables en Pseudocódigo

En PSeInt, el programa inicia con la sentencia **Algoritmo** **EvaluacionViabilidadTecnica** y la definición explícita de tipos de datos. La variable **nombreProyecto** se reserva como tipo **Character**, mientras que **puntajeRequisitos**, **puntajeArquitectura** y **promedio** se declaran como tipo **Real** para admitir cálculos decimales. En la Figura 4 se observa este bloque inicial.

Figura 4

Declaración del Algoritmo y Definición de Variables en PSeInt

```

1  Algoritmo EvaluacionViabilidadIecnica
2
3
4      Definir nombreProyecto Como Caracter
5      Definir puntajeRequisitos, puntajeArquitectura, promedio Como Real
6
7

```

Nota. Captura de pantalla del entorno PSeInt con el encabezado del algoritmo y la declaración formal de variables.

Solicitud de Datos y Cálculo del Promedio en Pseudocódigo

Posterior a la declaración de variables, el algoritmo solicita los datos de entrada al usuario mediante instrucciones **Escribir** y almacena las respuestas con la instrucción **Leer**. Primero se captura el nombre del proyecto estudiantil y la calificación de requisitos técnicos (Figura 5).

Figura 5

Instrucciones de Solicitud y Lectura de Datos Iniciales en PSeInt

```

Escribir "Ingrese el nombre del proyecto estudiantil:"
Leer nombreProyecto

Escribir "Ingrese el puntaje de requisitos técnicos (0 a 100):"
Leer puntajeRequisitos

```

Nota. Captura de pantalla del pseudocódigo que solicita y almacena el nombre del proyecto y la primera calificación técnica.

Seguidamente, se realiza la lectura del puntaje de viabilidad de la arquitectura de software. Con ambas variables en memoria, se ejecuta la operación de asignación `promedio <- (puntajeRequisitos + puntajeArquitectura) / 2`, tal como se documenta en la Figura 6.

Figura 6

Lectura del Puntaje de Arquitectura y Cálculo Aritmético del Promedio en PSeInt

```

14  Escribir "Ingrese el puntaje de viabilidad de la arquitectura (0 a 100):"
15  Leer puntajeArquitectura
16
17
18  promedio ← (puntajeRequisitos + puntajeArquitectura) / 2

```

Nota. Captura de pantalla de la captura del segundo criterio y la fórmula matemática de asignación del promedio.

Condición de Decisión en Pseudocódigo

La toma de decisiones se resuelve con la estructura de control Si-Entonces-Sino. El operador lógico de conjunción Y obliga a que se cumplan de manera simultánea las tres restricciones: promedio ≥ 75 Y puntajeRequisitos > 60 Y puntajeArquitectura > 60 . En la Figura 7 se aprecia el bloque de bifurcación condicional.

Figura 7

Estructura Condicional Compuesta y Generación de Mensajes en PSeInt

```

20
21  Si promedio ≥ 75 Y puntajeRequisitos > 60 Y puntajeArquitectura > 60 Entonces
22
23      Escribir "El proyecto ", nombreProyecto, " es VIABLE técnicamente. Promedio obtenido: ", promedio
24  SiNo
25
26      Escribir "El proyecto ", nombreProyecto, " REQUIERE AJUSTES para ser viable. Promedio obtenido: ", promedio
27  FinSi

```

Nota. Captura de pantalla de la estructura lógica Si-Entonces-Sino en PSeInt para clasificar la viabilidad técnica.

Representación Gráfica mediante Diagrama de Flujo

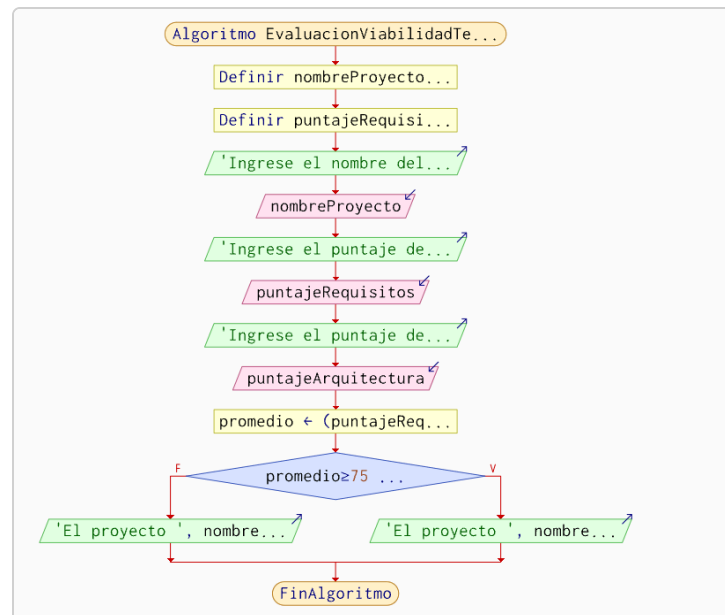
La modelación gráfica de la solución algorítmica permite visualizar la secuencia de control, los flujos de información y los puntos de bifurcación condicional de acuerdo con la notación estándar ANSI para diagramas de flujo (Vásquez, 2012).

Diagrama de Flujo en PSeInt

El diagrama de flujo inicia con el nodo terminal de inicio, continúa con los bloques rectangulares de definición de memoria, prosigue con los paralelogramos de entrada/salida y procesamiento aritmético, y desemboca en el rombo de decisión lógica compuesto, desde el cual se canalizan las dos alternativas de salida hacia el nodo terminal de fin. La Figura 8 presenta el diagrama generado en PSeInt.

Figura 8

Diagrama de Flujo de la Solución Algorítmica en PSeInt



Nota. Representación gráfica normalizada del flujo de control y las operaciones del algoritmo elaborada en PSeInt.

Pruebas de Escritorio y Ejecución en PSeInt

Con el propósito de verificar el comportamiento dinámico del pseudocódigo en PSeInt, se llevaron a cabo las pruebas de escritorio homólogas a las realizadas en el entorno Python.

Prueba de Escritorio 1: Proyecto con Requerimiento de Ajustes

En el primer escenario se ingresaron los valores: nombre = "notas", requisitos = 34 y arquitectura = 80. Con un promedio resultante de 57, la condición no se satisfizo, derivando el flujo hacia la salida de ajustes (Figura 9).

Figura 9

Evidencia de Ejecución 1 en PSeInt: Proyecto Requiere Ajustes

```
*** Ejecución Iniciada. ***  
Ingrese el nombre del proyecto estudiantil:  
> notas  
Ingrese el puntaje de requisitos técnicos (0 a 100):  
> 34  
Ingrese el puntaje de viabilidad de la arquitectura (0 a 100):  
> 80  
El proyecto notas REQUIERE AJUSTES para ser viable. Promedio obtenido: 57  
*** Ejecución Finalizada. ***
```

Nota. Captura de pantalla de la terminal de PSeInt donde se verifica el rechazo preventivo por puntaje individual y promedio insuficiente.

Prueba de Escritorio 2: Proyecto con Resultado Viable

En el segundo escenario se ingresaron los valores: nombre = "notas", requisitos = 80 y arquitectura = 90. Con un promedio obtenido de 85, la expresión lógica fue verdadera en su totalidad, confirmando la viabilidad técnica del proyecto (Figura 10).

Figura 10

Evidencia de Ejecución 2 en PSeInt: Proyecto Viable Técnicamente

```
*** Ejecución Iniciada. ***
Ingrese el nombre del proyecto estudiantil:
> notas
Ingrese el puntaje de requisitos técnicos (0 a 100):
> 80
Ingrese el puntaje de viabilidad de la arquitectura (0 a 100):
> 90
El proyecto notas es VIABLE técnicamente. Promedio obtenido: 85
*** Ejecución Finalizada. ***
```

Nota. Captura de pantalla de la terminal de PSeInt donde se constata la emisión del dictamen de viabilidad técnica satisfactoria.

Conclusiones

La implementación dual del algoritmo en lenguaje Python y en el entorno pedagógico PSeInt demostró la versatilidad de la lógica computacional estructurada para estandarizar procesos de toma de decisiones. Tanto la versión en Python como la representación en pseudocódigo y diagrama de flujo preservan una equivalencia semántica total, ratificando que un diseño algorítmico riguroso es independiente del lenguaje objetivo.

La inclusión concurrente de un umbral global ponderado y filtros de suficiencia individual para cada criterio técnico evaluado garantiza un control de calidad efectivo. Las evidencias de ejecución y pruebas de escritorio en ambas plataformas validaron la precisión, determinismo y cumplimiento integral de las directrices formativas del SENA.

Referencias

- Aguilar Joyanes, L. (2008). *Fundamentos de programación: Algoritmos, estructura de datos y objetos* (4.^a ed.). McGraw-Hill Interamericana.
- Servicio Nacional de Aprendizaje. (2026). *Guía de aprendizaje 01: Planeación y algoritmos*. SENA.
- Sierra, F. J. (2000). *C/C++ curso de programación*. Alianza Editorial.
- <https://www.elibronet.bdigital.sena.edu.co/es/ereader/senavirtual/50221?page=1>
- Vásquez, J. (2012). *Análisis y diseño de algoritmos*. Red Tercer Milenio.